Numerical Linear Algebra for Computational Sciences and Information Engineering
CITHN2006

Arnoldi and Lanczos Procedures

by Nicolas Venkovic

Chair of Computational Mathematics
School of Computation, Information and Technology (CIT)
Technical University of Munich, Germany

1. *Implement MGS and CGS2-based Arnoldi procedures.*

[1]:
```
using LinearAlgebra, Printf, ArnoldiMethod, SparseArrays
using MatrixMarket: mmread
```

[2]:
```
function MGS_arnoldi(A, q1::Vector{Float64}, k::Int)
  n, _ = size(A)
  Q = zeros(Float64, (n, k + 1))
  q = zeros(Float64, n)
  H = zeros(Float64, (k + 1, k))
  w = zeros(Float64,n)
  Q[:, 1] = q1 ./ norm(q1)
  for j in 1:k
    w .= A * Q[:, j]
    for i in 1:j
      q .= Q[:, i]
      H[i, j] = q'w
      w -= H[i, j] .* q
    end
    H[j + 1, j] = norm(w)
    Q[:, j + 1] = w ./ H[j + 1, j]
  end
  return Q, H
end;
```

[3]:
```
function CGS2_arnoldi(A, q1::Vector{Float64}, k::Int)
  n, _ = size(A)
  Q = zeros(Float64, (n, k + 1))
  H = zeros(Float64, (k + 1, k))
```

```
  h = zeros(Float64, k + 1)
  w = zeros(Float64, n)
  Q[:, 1] = q1 ./ norm(q1)
  for j in 1:k
    w .= A * Q[:, j]
    h .= Q'w
    h[j+1:k+1] .= 0.
    H[:, j] = h
    w -= Q * h
    h .= Q'w
    h[j+1:k+1] .= 0.
    w -= Q * h
    H[j + 1, j] = norm(w)
    Q[:, j + 1] = w ./ H[j + 1, j]
  end
  return Q, H
end;
```

2. *For both methods, check how close $Q_{k+1}^T A Q_k$ is to your Hessenberg matrix, and with what accuracy does your Arnoldi relation $AQ_k = Q_{k+1}\underline{H}_k$ hold. Quantify also the loss of orthgonality of the basis, and compare runtimes. For all this, use the matrix* `G3 circuit` *from the SuiteSparse collection.*

[4]:
```
A = mmread("matrices/G3_circuit.mtx");
n, _ = size(A);
```

[5]:
```
k = 100;
```

[6]:
```
Q, H = @time MGS_arnoldi(A, rand(n), k);
@printf "||Qjp1 * A * Qj - H|| = %.2E, ||A * Qj - Qjp1 * H|| = %.2E, ||I -␣
 ↪Q'Q|| = %.2E" norm(Q'A * Q[:, 1:k] - H) norm(A * Q[:, 1:k] - Q * H) norm(I -␣
 ↪Q'Q)
```

```
 33.928746 seconds (1.68 M allocations: 183.825 GiB, 5.45% gc time, 1.48%
compilation time)
||Qjp1 * A * Qj - H|| = 2.64E-07, ||A * Qj - Qjp1 * H|| = 5.44E-11, ||I - Q'Q||
= 1.97E-11
```

[7]:
```
Q, H = @time CGS2_arnoldi(A, rand(n), k);
@printf "||Qjp1 * A * Qj - H|| = %.2E, ||A * Qj - Qjp1 * H|| = %.2E, ||I -␣
 ↪Q'Q|| = %.2E" norm(Q'A * Q[:, 1:k] - H) norm(A * Q[:, 1:k] - Q * H) norm(I -␣
 ↪Q'Q)
```

```
 14.425879 seconds (391.07 k allocations: 9.515 GiB, 2.53% gc time, 1.46%
compilation time)
||Qjp1 * A * Qj - H|| = 2.94E-10, ||A * Qj - Qjp1 * H|| = 8.41E-11, ||I - Q'Q||
= 1.68E-14
```

3. *Implement a Rayleigh-Ritz procedure on top of your CGS2-based Arnoldi process. For the matrix*

`nxp1` from the SuiteSparse collection, solve for the 5 largest eigenpairs using the reduced expressions derived in class to monitor convergence of residuals. Verify your residual estimates and compare your converged eigenpairs with those obtained using `ArnoldiMethod.jl`.

[8]:
```
function RR_CGS2_arnoldi(A, q1::Vector{Float64}, k::Int, nev::Int, tol::Float64)
  n, _ = size(A)
  Q = zeros(Float64, (n, k + 1))
  H = zeros(Float64, (k + 1, k))
  h = zeros(Float64, k + 1)
  w = zeros(Float64, n)
  Q[:, 1] = q1 ./ norm(q1)
  j = 0
  res = 1.; vals = nothing; vecs = nothing; order = nothing
  while (maximum(res) > tol && j < k)
    j += 1
    w .= A * Q[:, j]
    h .= Q'w
    h[j+1:k+1] .= 0.
    H[:, j] = h
    w -= Q * h
    h .= Q'w
    h[j+1:k+1] .= 0.
    w -= Q * h
    H[j + 1, j] = norm(w)
    Q[:, j + 1] = w ./ H[j + 1, j]
    vals, vecs = eigen(H[1:j, 1:j], sortby=norm)
    order = (j:-1:j-min(nev,j)+1)
    res = norm(H[j + 1, j]) .* norm.(vecs[j, order]) ./ norm.(vals[order])
    println(res)
  end
  return res, vals[order], Q[:, 1:j] * vecs[:, order]
end;
```

[9]:
```
A = mmread("matrices/nxp1.mtx");
n, _ = size(A);
```

[10]:
```
k = 100;
nev = 5;
tol = 1e-6;
res, vals, Y = RR_CGS2_arnoldi(A, rand(n), k, nev, tol);
```

```
[7.100034571052361]
[0.8498509454621034, 9.404543582286799]
[0.4751209587742315, 0.9423676950721311, 9.583334946021248]
[0.22222443849716222, 0.43968181419463154, 0.8080800522818986, 7.41670463131695]
[0.09576434535953417, 0.3229745988145619, 0.6857967662153509, 1.132013917684396,
8.442028095266867]
[0.06253977287472226, 0.26646760756935095, 0.30931882615596695,
```

```
0.6197298511067076, 1.3850263793087756]
[0.02810806365835276, 0.11388276476995156, 0.19190636162038754,
0.359368507730765, 0.5055043161098917]
[0.009019247957523454, 0.047998192495012144, 0.16467851036334816,
0.24709085768587336, 0.4090607148590571]
[0.0020612239713248778, 0.015529298774124485, 0.09740081247127964,
0.1408226387442795, 0.25352032077860087]
[0.0005467973859679752, 0.00584163675196506, 0.07204205501933245,
0.14944013966799727, 0.26579935632612023]
… (24 lines omitted) …
[1.0695169391580027e-20, 5.156967141898175e-16, 0.003510677425396449,
0.00787400877428344, 0.0030192627228792924]
[1.768328071942569e-21, 1.1343023086966943e-16, 0.0014245576673559597,
0.002736847094297851, 0.001060804857784495]
[3.1935695440761662e-22, 2.7182479953154662e-17, 0.0005836814792182028,
0.0010980207594803323, 0.0004307907562834162]
[4.80646674854587e-23, 5.4326054806953775e-18, 0.00019622924848446602,
0.0003684410621600 2675, 0.00014624663673290647]
[7.619075793605602e-24, 1.2572362364936372e-18, 6.278495629705936e-5,
0.00011801954832996158, 4.730036087733101e-5]
[2.4545062941350342e-24, 2.5305390519674184e-19, 2.4245896673316133e-5,
4.5649827417553286e-5, 1.8491262595003677e-5]
[2.427271737996699e-25, 5.951527732496336e-20, 9.240538586806636e-6,
1.7428400783197116e-5, 7.139538091803503e-6]
[1.8988840424635765e-25, 9.590500744605628e-21, 3.493897329060492e-6,
6.601395596933593e-6, 2.7350038861154785e-6]
[8.189184807643802e-27, 3.198902074769496e-21, 1.2413964402818859e-6,
2.3494892132205105e-6, 9.84061512071598e-7]
[6.881454212756889e-26, 5.6260304665140636e-21, 4.4145463872043267e-7,
8.368874468699641e-7, 3.5425511330528444e-7]
```

[11]:
```
res2 = [norm(A * Y[:, i] - vals[i] .* Y[:, i]) / norm(vals[i]) for i in 1:nev];
maximum(res - res2)
```

[11]: 2.569209747699903e-18

[12]:
```
decomp, hist = partialschur(A, nev=nev);
vals2, Y2 = partialeigen(decomp);
```

[13]:
```
maximum(abs.(vals2[end:-1:1] .- vals))
```

[13]: 3.753370947379153e-10

[14]:
```
norm(abs.(Y'Y2[:, end:-1:1]) - I)
```

[14]: 1.6390760166107417e-9

[15]:
```
vals
```

[15]:
```
5-element Vector{Float64}:
 414.73857473466364
 341.53916763358745
 257.3852366214112
 257.18132297527734
 255.88531378923255
```

4. *Implement a haronic Ritz procedure on top of your CGS2-based Arnoldi process. Still using the* ***nxp1*** *matrix, solve for the 2 eigenpairs closest to* $\sigma = 150$ *using the reduced expressions derived in class to monitor convergence of residuals. Verify your residual estimates.*

[16]:
```
function HR_CGS2_arnoldi(A, q1::Vector{Float64}, k::Int, nev::Int, sigma::
↪ComplexF64, tol::Float64)
 n, _ = size(A)
 Q = zeros(Float64, (n, k + 1))
 H = zeros(Float64, (k + 1, k))
 h = zeros(Float64, k + 1)
 w = zeros(Float64, n)
 Q[:, 1] = q1 ./ norm(q1)
 j = 0
 res = 1.; rho = nothing; vecs = nothing
 while (maximum(res) > tol && j < k)
   j += 1
   w .= A * Q[:, j]
   h .= Q'w
   h[j+1:k+1] .= 0.
   H[:, j] = h
   w -= Q * h
   h .= Q'w
   h[j+1:k+1] .= 0.
   w -= Q * h
   H[j + 1, j] = norm(w)
   Q[:, j + 1] = w ./ H[j + 1, j]
   e_j = zeros(Float64, j); e_j[j] = 1.
   f = (H[1:j, 1:j] - sigma * I)' \ e_j
   G = H[1:j, 1:j] + norm(H[j + 1, j])^2 .* f * e_j'
   vals, vecs = eigen(G)
   order = sortperm(norm.(vals .- sigma))[1:minimum([j, nev])]
   vals = vals[order]
   vecs = vecs[:, order]
   rho = [vals[i] - norm(H[j + 1, j])^2 * vecs[:, i]'f * vecs[j, i] for i in 1:
↪minimum([j, nev])]
   res = sqrt.(real.((vals .- rho) .* conj.(rho .- sigma)))
   res ./= norm.(rho)
   #hres = (vals .- rho) .* conj.(vals .- sigma)
   println(res)
 end
```

```
  return res, rho, Q[:, 1:j] * vecs
end;
```

[17]:
```
k = 500
nev = 2;
sigma = 150. + 0im;
res, vals, Y = HR_CGS2_arnoldi(A, rand(n), k, nev, sigma, tol);
```

```
[7.122928138347147]
[1.1353390922098061, 1.4869555086965558]
[0.7481103868599336, 11.64375797916595]
[0.7483524406528268, 12.086895229077067]
[0.48704689687812475, 0.2400420566737263]
[0.23729154120433885, 0.4902953224626831]
[0.1865525683896828, 0.3262274831235198]
[0.1570236086264957, 0.3097151764527024]
[0.10142366177487894, 0.20865724262100252]
[0.1047264642330438, 0.20811029677275114]
… (214 lines omitted) …
[6.5212041723962e-6, 5.1198738090588106e-5]
[5.067512481228536e-6, 3.964417947070777e-5]
[3.7536490170743595e-6, 2.9100994118685784e-5]
[2.4616513496084925e-6, 1.8848022634085056e-5]
[1.613973222454422e-6, 1.2226592955879023e-5]
[1.012906731422865e-6, 7.591994695493642e-6]
[5.582253248499918e-7, 4.141293944414835e-6]
[3.0276825996418035e-7, 2.2285618943081114e-6]
[1.7160962478123559e-7, 1.2543545284952375e-6]
[9.525835318490645e-8, 6.913572547902072e-7]
```

[18]:
```
res2 = [norm(A * Y[:, i] - vals[i] .* Y[:, i]) / norm(vals[i]) for i in 1:nev];
maximum(res - res2)
```

[18]:
```
2.708846365779424e-13
```

[19]:
```
vals
```

[19]:
```
2-element Vector{ComplexF64}:
  151.0742742332792 + 2.905218957062148e-16im
 151.37728530083484 - 4.4022010489296194e-16im
```

[20]:
```
res
```

[20]:
```
2-element Vector{Float64}:
 4.863132420582555e-7
 7.744570224631999e-7
```

5. *Implement the Lanczos algorithm. Check how close $Q_{k+1}AQ_k$ is to your tridiagonal matrix, and with what accuracy does your Lanczos relation $AQ_k = Q_{k+1}\underline{T}_k$ hold. Quantify also the loss of*

orthogonality of the basis. For all this, use the matrix **Kuu** *from the SuiteSparse collection.*

[22]:
```
function lanczos(A, q1::Vector{Float64}, k::Int)
  n, _ = size(A)
  Q = zeros(Float64, (n, k + 1))
  v = zeros(Float64, n)
  T = zeros(Float64, k + 1, k + 1)
  Q[:, 1] = q1 ./ norm(q1)
  for j in 1:k
    v = A * Q[:, j]
    T[j, j] = Q[:, j]'v
    v = v - T[j, j] * Q[:, j]
    if (j > 1)
      v -= T[j, j - 1] * Q[:, j - 1]
    end
    T[j + 1, j] = norm(v); T[j, j + 1] = T[j + 1, j]
    Q[:, j + 1] = v ./ T[j + 1, j]
  end
  return Q, T[1:k+1, 1:k]
end;
```

[23]:
```
A = mmread("matrices/Kuu.mtx");
n, _ = size(A);
```

[24]:
```
k = 100;
```

[25]:
```
Q, T = @time lanczos(A, rand(n), k);
@printf "||Qjp1 * A * Qj - T|| = %.2E, ||A * Qj - Qjp1 * T|| = %.2E, ||I -␣
 ↪Q'Q|| = %.2E" norm(Q'A * Q[:, 1:k] - T) norm(A * Q[:, 1:k] - Q * T) norm(I -␣
 ↪Q'Q)
```

```
  0.178792 seconds (95.04 k allocations: 63.991 MiB, 83.71% compilation time)
||Qjp1 * A * Qj - T|| = 7.76E-10, ||A * Qj - Qjp1 * T|| = 2.45E-14, ||I - Q'Q||
= 1.60E-11
```

6. *Implement a Rayleigh-Ritz procedure on top of your Lanczos process. For the matrix* **Kuu** *from the SuiteSparse collection, solve for the 5 largest eigenpairs using the reduced expressions derived in class to monitor convergence of residuals. Verify your residual estimates.*

[26]:
```
function RR_lanczos(A, q1::Vector{Float64}, k::Int, nev::Int, tol::Float64)
  n, _ = size(A)
  Q = zeros(Float64, (n, k + 1))
  v = zeros(Float64, n)
  T = zeros(Float64, k + 1, k + 1)
  vals = zeros(Float64, nev, k)
  Q[:, 1] = q1 ./ norm(q1)
  j = 0
  res = 1.; vecs = nothing; order = nothing
  while (maximum(res) > tol && j < k)
```

```
    j += 1
    v = A * Q[:, j]
    T[j, j] = Q[:, j]'v
    v = v - T[j, j] * Q[:, j]
    if (j > 1)
      v -= T[j, j - 1] * Q[:, j - 1]
    end
    T[j + 1, j] = norm(v); T[j, j + 1] = T[j + 1, j]
    Q[:, j + 1] = v ./ T[j + 1, j]
    _vals, vecs = eigen(T[1:j, 1:j], sortby=norm)
    order = (j:-1:j-min(nev,j)+1)
    vals[1:minimum([j, nev]), j] = _vals[order]
    res = norm(T[j + 1, j]) .* norm.(vecs[j, order]) ./ norm.(_vals[order])
    println(res)
  end
  return res, vals[:, 1:j], Q[:, 1:j] * vecs[:, order]
end;
```

[27]:
```
k = 500;
nev = 5;
res, vals, Y = RR_lanczos(A, rand(n), k, nev, tol);
```

```
[2.4238361249552196]
[0.4187032912005488, 3.7701578010711407]
[0.2519120616083001, 0.5670520717332632, 5.024604945755622]
[0.13240053279218578, 0.25953163961230435, 0.5725039097961709,
5.720586138258274]
[0.06042766771842754, 0.17619526152890636, 0.405524378436357,
0.7512831805062368, 7.525639246550031]
[0.058226437432262544, 0.25425453681439203, 0.2619858343176535,
0.3904697523294235, 0.7220137561280378]
[0.04421501570835561, 0.11323625370082072, 0.10908180692180389,
0.27862759392154685, 0.48477827005955737]
[0.03828912465586239, 0.08709400211695446, 0.2116072407876117,
0.2955111739559017, 0.28875196854039126]
[0.036983358407989525, 0.07906506019582306, 0.1782052387163235,
0.10631762241161241, 0.27860573730488936]
[0.03591669966441124, 0.06601783370625834, 0.10875503166528246,
0.130778593897926, 0.2399614194504613]
… (209 lines omitted) …
[3.996130724365026e-12, 1.8844325415600984e-11, 5.163334814819838e-9,
1.0662421895733489e-7, 6.620123443879452e-6]
[3.719762083788552e-12, 1.779066086431981e-11, 4.998687227675075e-9,
1.0485176795793258e-7, 6.62431641211837e-6]
[2.311540200443446e-12, 1.1223788845800109e-11, 3.2402072748549513e-9,
6.913180522190509e-8, 4.4514476273152245e-6]
[1.4770619554687922e-12, 7.2425955751388594e-12, 2.1274884895458834e-9,
4.588139712804389e-8, 2.9895808352214692e-6]
```

```
[1.146461195178387e-12, 5.680432903223477e-12, 1.6992255881735562e-9,
3.705263383074943e-8, 2.4434480918439936e-6]
[1.038850545889548e-12, 5.2251955938505e-12, 1.604745495506993e-9,
3.5558342860019064e-8, 2.3861312385983055e-6]
[6.204213552151253e-13, 3.1659867913098825e-12, 9.975859948499496e-10,
2.2457838043622157e-8, 1.5334848591371163e-6]
[4.769947930326068e-13, 2.4590403040327843e-12, 7.888495080437942e-10,
1.7954597813825012e-8, 1.2407308835086352e-6]
[4.1970914412505163e-13, 2.198100139852782e-12, 7.250113765246204e-10,
1.678518827108111e-8, 1.1816821545210785e-6]
[2.752547906763983e-13, 1.4625843530516253e-12, 4.950283654391378e-10,
1.1645894535960196e-8, 8.344897747731265e-7]
```

[28]:
```
res2 = [norm(A * Y[:, i] - vals[i, end] .* Y[:, i]) / norm(vals[i, end]) for i␣
 ↪in 1:nev];
maximum(res - res2)
```

[28]:
```
3.673515717792896e-18
```

[29]:
```
vals[:, end]
```

[29]:
```
5-element Vector{Float64}:
 54.08205599641256
 53.98165804506268
 53.81549645283013
 53.72040205150498
 53.62254145467433
```

7. *Implement a harmonic Ritz procedure on top of your Lanczos process. For the matrix* **Kuu** *from the SuiteSparse collection, solve for the 2 eigenpairs closest to* $\sigma = 25$ *using the reduced expressions derived in class to monitor convergence of residuals. Verify your residual estimates.*

[30]:
```
function HR_lanczos(A, q1::Vector{Float64}, k::Int, nev::Int, sigma::Float64,␣
 ↪tol::Float64)
  n, _ = size(A)
  Q = zeros(Float64, (n, k + 1))
  v = zeros(Float64, n)
  T = zeros(Float64, k + 1, k + 1)
  rhos = zeros(ComplexF64, nev, k)
  Q[:, 1] = q1 ./ norm(q1)
  j = 0
  res = 1.; vecs = nothing; order = nothing
  while (maximum(res) > tol && j < k)
    j += 1
    v = A * Q[:, j]
    T[j, j] = Q[:, j]'v
    v = v - T[j, j] * Q[:, j]
    if (j > 1)
```

```
      v -= T[j, j - 1] * Q[:, j - 1]
    end
    T[j + 1, j] = norm(v); T[j, j + 1] = T[j + 1, j]
    Q[:, j + 1] = v ./ T[j + 1, j]
    e_j = zeros(Float64, j); e_j[j] = 1.
    f = (T[1:j, 1:j] - sigma * I) \ e_j
    G = T[1:j, 1:j] + T[j + 1, j]^2 .* f * e_j'
    vals, vecs = eigen(G)
    order = sortperm(norm.(vals .- sigma))[1:minimum([j, nev])]
    vals = vals[order]
    vecs = vecs[:, order]
    rho = [vals[i] - T[j + 1, j]^2 * vecs[:, i]'f * vecs[j, i] for i in 1:
 ↪minimum([j, nev])]
    res = sqrt.(real.((vals .- rho) .* conj.(rho .- sigma)))
    res ./= norm.(rho)
    rhos[1:minimum([j, nev]), j] = rho
    println(res)
  end
  return res, rhos[:, 1:j], Q[:, 1:j] * vecs
end;
```

[31]:
```
k = 1_000;
nev = 2;
sigma = 25.;
res, vals, Y = HR_lanczos(A, rand(n), k, nev, sigma, tol);
```

```
[2.439319348719891]
[2.892201045837972, 0.4098688733540916]
[0.5204015840666399, 0.25088907329049814]
[0.2233159879615806, 0.5016648990229926]
[0.15003873490534395, 0.35313584158682476]
[0.15183209032649503, 0.2959576158648413]
[0.09981392486669959, 0.2260131981805472]
[0.19462534659117411, 0.11706931414454619]
[0.17542671801403115, 0.0914910165301863]
[0.14067712847386124, 0.10782306153407911]
… (1791 lines omitted) …
[1.1796240655294799e-7, 1.8820613103964957e-6]
[1.1394598011792546e-7, 1.8240707290176903e-6]
[1.0515566751316999e-7, 1.68218425332485e-6]
[1.0354725881909541e-7, 1.6592648045258547e-6]
[9.80353876247863e-8, 1.568948733749702e-6]
[9.380770369002223e-8, 1.5062540166081488e-6]
[8.46723680244122e-8, 1.359276258715792e-6]
[8.02698035361406e-8, 1.2916811628345033e-6]
[6.858433277747964e-8, 1.1019612801943533e-6]
[6.731067445531585e-8, 1.0829061359834529e-6]
```

[32]:
```
res2 = [norm(A * Y[:, i] - vals[i, end] .* Y[:, i]) / norm(vals[i, end]) for i
 ↪in 1:nev];
maximum(res - res2)
```

[32]:
```
-1.442512355767124e-15
```

[33]:
```
vals[:, end]
```

[33]:
```
2-element Vector{ComplexF64}:
 25.005559914043662 + 0.0im
  25.06155297525221 + 0.0im
```

[34]:
```
Y
```

[34]:
```
7102×2 Matrix{Float64}:
  0.001668     -0.0017046
  0.00334998   -0.00421567
  0.00610722   -0.010005
 -0.00360319    0.00390963
 -0.0101473     0.015974
 -0.00257295    0.00371061
  0.00166767   -0.00151603
  0.00857564   -0.0103629
  0.0128581    -0.0192621
 -0.00519509    0.0056193
 -0.0140194     0.0192927
 -0.00487762    0.00556106
 -0.000283408   0.00149261

 -4.47564e-6    6.20109e-6
  6.92231e-7   -1.14319e-6
  4.98351e-6   -6.342e-6
  1.06519e-6   -1.06987e-6
 -3.64721e-7    2.96701e-7
 -8.07335e-7    1.02475e-6
 -2.39733e-6    3.9843e-6
 -2.44157e-7   -3.30857e-8
  2.97405e-6   -3.75317e-6
  1.28912e-6   -1.39285e-6
 -2.34245e-7    2.58633e-7
 -4.16712e-7    5.12173e-7
```